

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №2
з дисципліни: «Архітектура програмного забезпечення»
з теми: «Мобільний застосунок»

Виконав:

ст. гр. ПЗПІ-23-8

Потурайко Н. І.

Перевірив:

асистент кафедри ПІ

Дашенков Д. С.

2 МОБІЛЬНИЙ ЗАСТОСУНОК

2.1 Мета роботи

Метою даної лабораторної роботи є проектування та повна програмна реалізація мобільного клієнта для системи інтелектуального моніторингу Aquila на платформі Android. Основним завданням є створення надійного інструменту для віддаленого контролю параметрів мікроклімату та управління підсистемою безпеки. Застосунок має забезпечувати розмежування прав доступу між адміністратором та користувачем, реалізовувати механізм отримання миттєвих сповіщень про критичні події через хмарні сервіси, а також надавати зручний інтерфейс для перегляду візуальних доказів тривожних ситуацій у вигляді знімків з камер спостереження.

2.2 Хід роботи

2.2.1 Прийняті інженерні рішення

2.2.1.1 Платформа та мова програмування

Для реалізації проєкту обрано нативну розробку на мові Kotlin під операційну систему Android. Такий вибір зумовлений потребою у високій продуктивності застосунку та необхідністю глибокої інтеграції з системними сервісами. Нативна реалізація дозволяє ефективно керувати фоновими процесами для отримання телеметрії та забезпечує стабільну роботу мережевих протоколів при прямій взаємодії з IoT обладнанням на базі ESP32.

2.2.1.2 Сучасний інтерфейс Jetpack Compose

Графічний інтерфейс користувача побудовано на базі декларативного фреймворку Jetpack Compose. Це дозволило відмовитися від застарілих XML-розміток на користь опису UI як набору функцій, що залежать від стану. Такий підхід значно пришвидшує розробку та дозволяє легко реалізувати складні візуальні ефекти Cyber-Minimalism, включаючи напівпрозорі картки пристроїв та динамічні графіки. Система автоматично перемальовує лише ті елементи, дані в яких змінилися, що позитивно впливає на енергоефективність пристрою.

2.2.1.3 Архітектурний шаблон MVVM

Для забезпечення чистоти коду та зручності тестування застосовано архітектуру Model-View-ViewModel. Шар Model відповідає за структуру даних телеметрії та взаємодію з базою PostgreSQL через API. Шар View представлений компонентами Compose, які не містять бізнес-логіки. ViewModel виступає посередником: вона отримує дані від репозиторіїв, обробляє їх та надає UI через об'єкти StateFlow. Це гарантує, що при повороті екрана або перемиканні між додатками стан моніторингу не буде втрачено.

2.2.1.4 Мережева інфраструктура та Retrofit

Взаємодія з серверною частиною на Node.js реалізована за допомогою бібліотеки Retrofit. Оскільки система працює з великою кількістю дрібних запитів телеметрії, налаштовано пул з'єднань через OkHttp клієнт. Для роботи з зображеннями, які надсилає ESP32 при детекції руху, використано бібліотеку Coil. Вона забезпечує ефективне кешування знімків та їх поступове завантаження, що важливо при нестабільному мобільному інтернеті.

2.2.1.5 Рольова модель та управління доступом

У застосунку впроваджено жорстку логіку розмежування функціоналу. Після авторизації додаток перевіряє роль у JWT токени. Якщо користувач є адміністратором, йому відкриваються додаткові екрани для CRUD операцій: створення кімнат, реєстрація пристроїв за MAC-адресою та управління обліковими записами. Звичайний користувач бачить лише результати моніторингу та алерти. Така реалізація підвищує безпеку системи, унеможливаючи випадкову зміну критичних налаштувань рядовими споживачами.

2.2.1.6 Безпека та менеджер сесій

Зберігання конфіденційних даних, таких як токени авторизації, реалізовано за допомогою EncryptedSharedPreferences. Це бібліотека, яка забезпечує апаратне шифрування даних на рівні ОС Android. Менеджер сесій автоматично оновлює

токени та перенаправляє користувача на екран входу в разі закінчення терміну дії сесії, що відповідає сучасним стандартам безпеки банківських та IoT додатків.

2.2.1.7 Теми та візуальна адаптація

Згідно з концепцією дизайну майбутнього, основною темою застосунку є Dark Mode. Це не лише естетичне рішення, а й технічна необхідність для зменшення навантаження на зір під час перегляду графіків у темний час доби. Застосунок підтримує повну інтернаціоналізацію, надаючи інтерфейс українською мовою. Форматування дат та чисел адаптується під регіональні налаштування смартфона користувача.

2.2.1.8 Хмарні сповіщення через Firebase

Для миттєвої реакції на події безпеки інтегровано Firebase Cloud Messaging. При виявленні руху IoT пристрій надсилає сигнал на бекенд, який ініціює push-сповіщення через хмару Google. Мобільний додаток обробляє ці повідомлення пріоритетно, дозволяючи користувачу миттєво відкрити знімок з місця події.

2.2.2 Будова застосунку та взаємодія з системою

2.2.2.1 Потоки даних з серверною частиною

Застосунок є активним споживачем REST API. При кожному відкритті екрана кімнати виконується запит на отримання останніх вимірювань. Дані про температуру та вологість приходять у форматі масивів об'єктів, які потім конвертуються в точки для лінійних графіків. Для оптимізації трафіку впроваджено механізм пагінації при перегляді великих журналів алертів та історії знімків.

2.2.2.2 Процес налаштування обладнання

Важливою функцією є реєстрація нових IoT пристроїв. Адміністратор використовує мобільний додаток для прив'язки фізичного модуля ESP32 до віртуальної кімнати в системі. Після введення MAC-адреси додаток відправляє запит на сервер, який перевіряє наявність пристрою в мережі та активує його профіль у базі даних PostgreSQL.

2.3 Висновки

У ході виконання лабораторної роботи було розроблено повнофункціональний мобільний Android-застосунок для системи Aquila. Використання мови Kotlin та фреймворку Jetpack Compose дозволило створити реактивний інтерфейс з високим рівнем відгуку. Архітектурне рішення на базі MVVM забезпечило стабільність роботи при обробці потокових даних телеметрії. Реалізована рольова модель доступу та система миттєвих сповіщень роблять застосунок надійним інструментом для адміністрування та моніторингу віддалених об'єктів. Проєкт повністю інтегрований з раніше розробленим бекендом та IoT клієнтом, що підтверджує цілісність архітектури всієї системи моніторингу.

ДОДАТОК А

Діаграма прецедентів

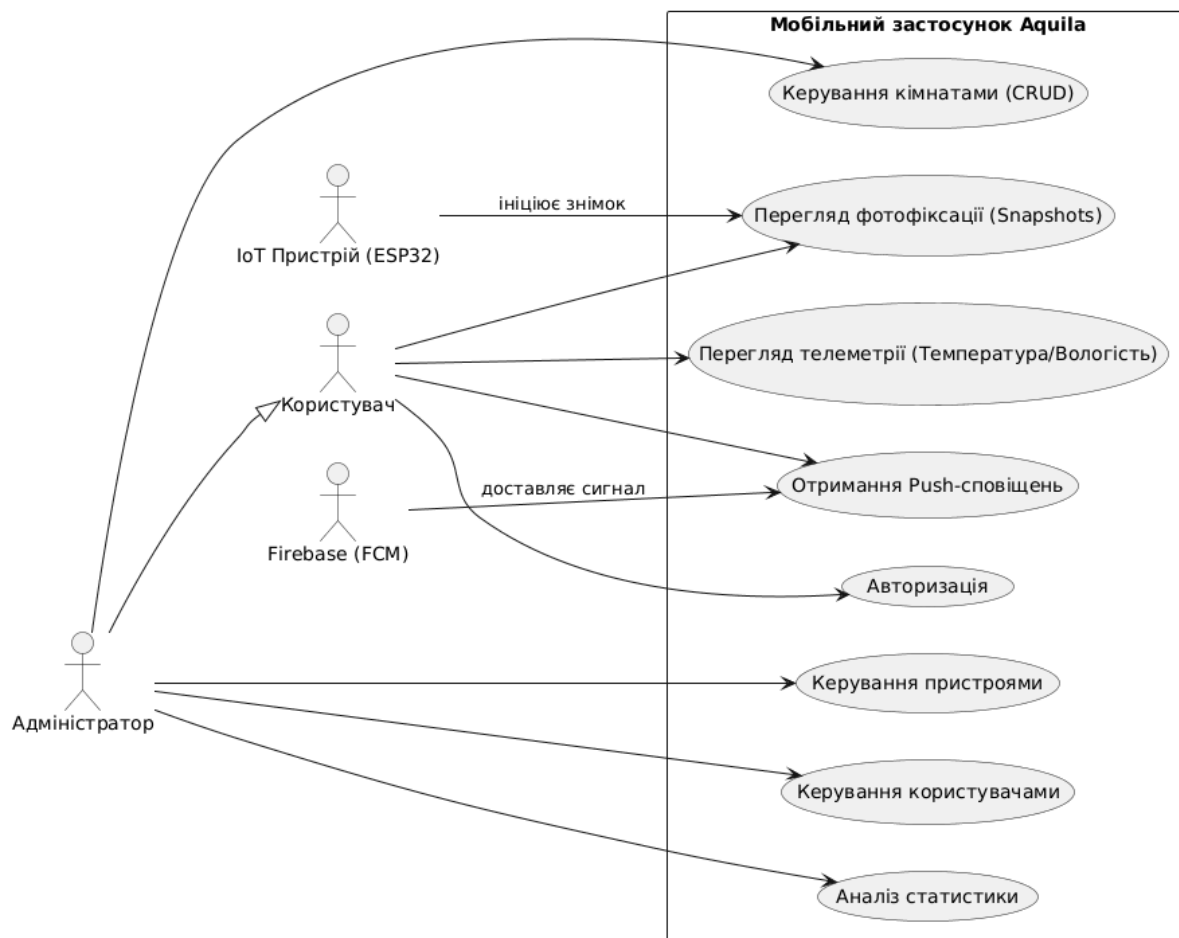


Рисунок А.1 – Діаграма прецедентів для мобільного застосунку

ДОДАТОК Б

Діаграма компонентів

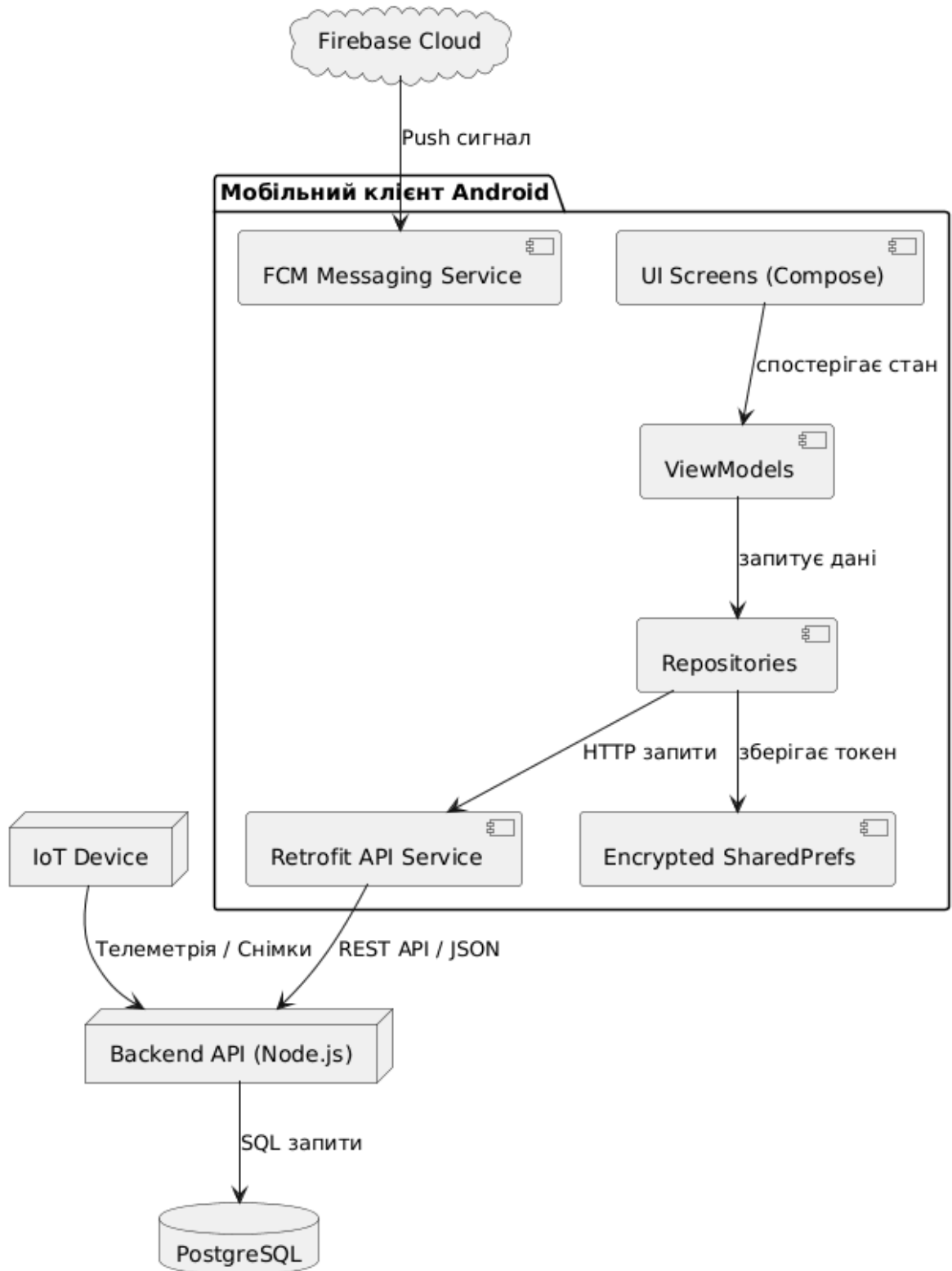


Рисунок Б.1 – Діаграма компонентів мобільного застосунку

ДОДАТОК В

Діаграма діяльності

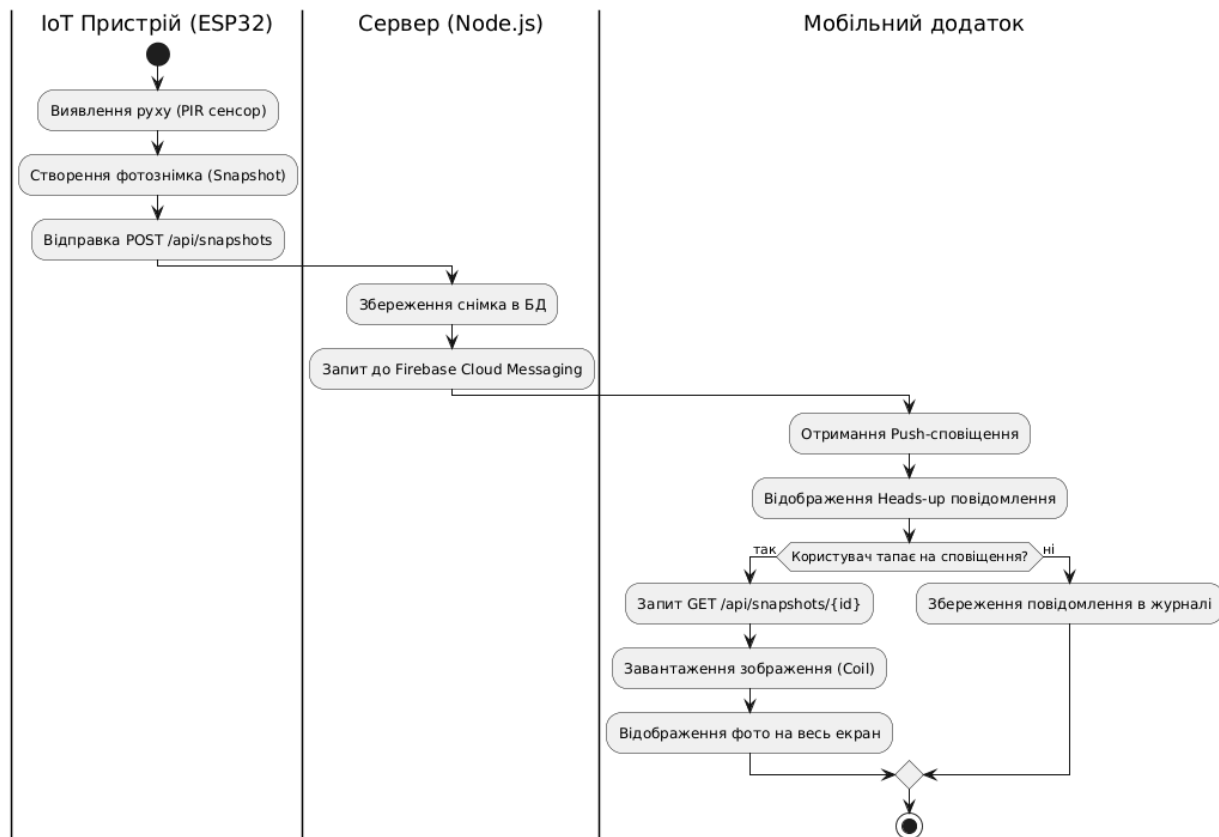


Рисунок В.1 – Діаграма діяльності для процесу отримання телеметрії та відображення графіків

ДОДАТОК Г

Діаграма станів

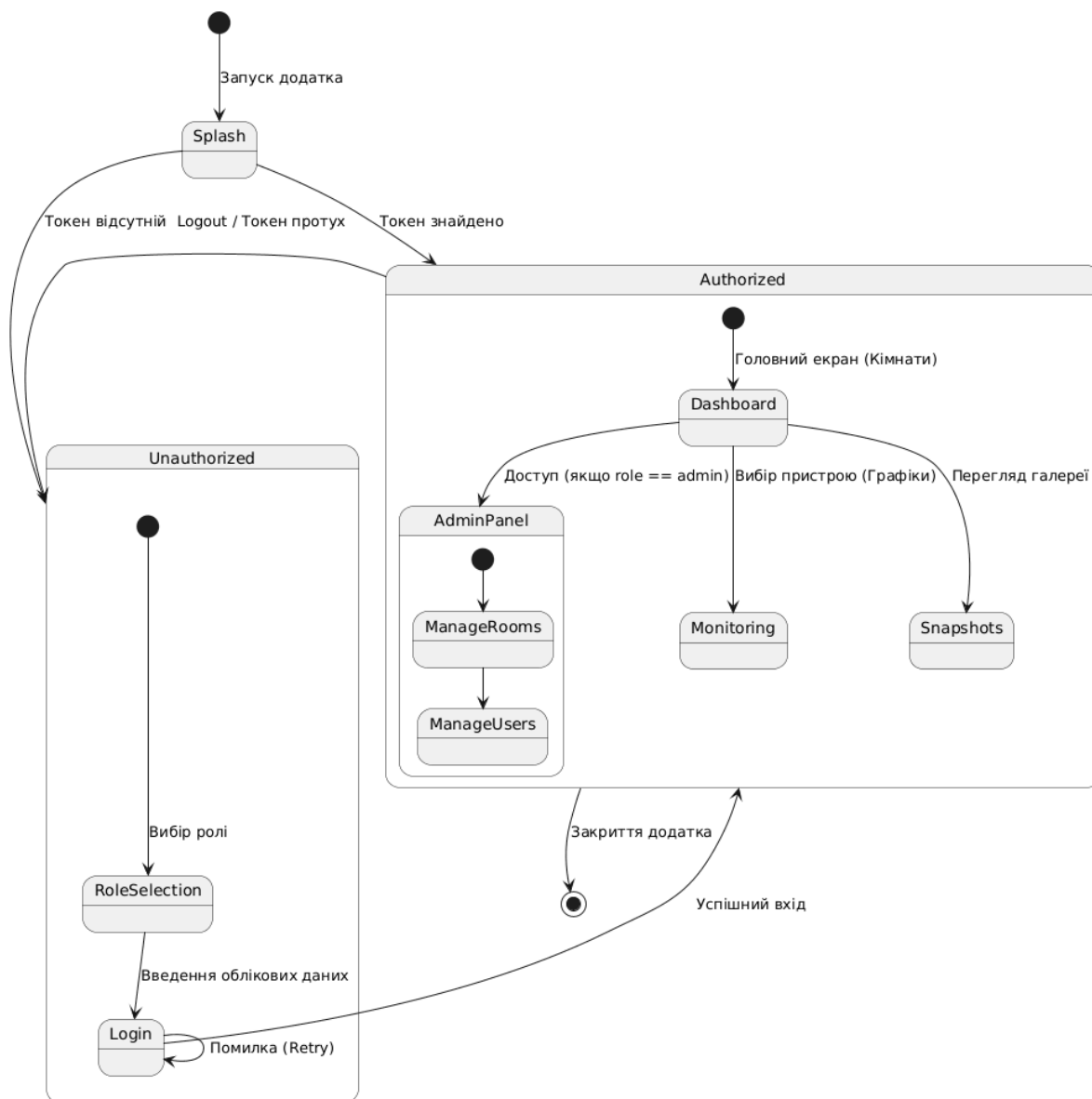


Рисунок Г.1 – Діаграма станів для екрану кімнати з моніторингом параметрів

ДОДАТОК Д
Репозиторій проекту

<https://github.com/NurePoturaikoNazar/apz-project>